

1. 简介

1.1. 总述

本库专为沁恒微电子 CH585 系列和 CH595 系列微控制器优化设计，提供高性能、低功耗的触摸按键检测功能。

1.2. 库使用流程

如图 1 所示，展示了触摸库的一个完整的最简使用流程。

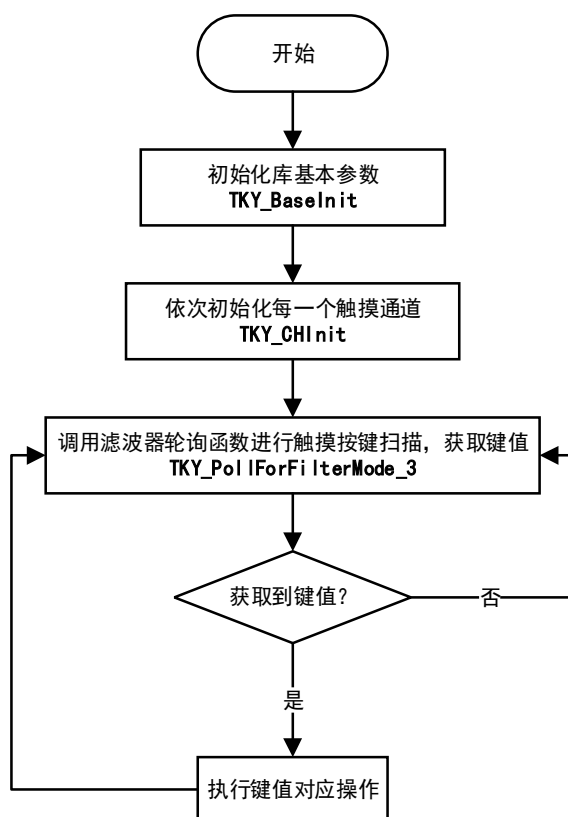


图 1 库使用流程图

2. TouchKey_CFG.h 配置文件说明

TouchKey_CFG.h 文件由上位调试工具生成，直接决定着产品的触摸性能。对于 PCB 设计、外壳及装配方式已确定的产品，此文件通常无需修改。

该文件也仅反映调试时的硬件环境，例如与上位机连接调试时硬件为裸 PCB 的状态，那么此时校准生成的 cfg 文件在 PCB 装配到产品外壳上时已不再适用。

2.1. 队列枚举定义

```
typedef enum _TKY_QUEUE_ID
{
    TKY_QUEUE_0 = 0,
    // ... 共 24 个队列 ID
    TKY_QUEUE_END
} TKY_QUEUE_ID;
```

- 定义了 24 个触摸通道队列标识符
- 用于区分不同的触摸通道

2.2. 核心配置宏定义

配置项	说明
TKY_FILTER_MODE	滤波模式设置
TKY_FILTER_GRADE	滤波等级
TKY_BASE_REFRESH_ON_PRESS	按下时基准刷新使能
TKY_BASE_UP_REFRESH_DOUBLE	按键释放基准跌落恢复的刷新加快倍数
TKY_BASE_DOWN_REFRESH_SLOW	按键触发基准向触发方向刷新抑制倍数
TKY_BASE_REFRESH_SAMPLE_NUM	基准刷新采样次数
TKY_SHIELD_EN	屏蔽功能使能
TKY_SINGLE_PRESS_MODE	单按键模式
TKY_TOUCH_QUEUE_NUM	触摸队列数量
TKY_SLEEP_QUEUE_NUM	休眠队列数量
TKY_MAX_QUEUE_NUM	最大队列数量(触摸+休眠)
TKY_MAX_NOISE_CH_COUNT	最大噪声通道计数
TKY_CX_CH_IDX	外接电容的 adc 通道索引

注：详细说明见 3.1 节

2.3. 通道参数配置宏

```
GEN_TKY_CH_INIT(qNum, chNum, hyswin, chBaseline, maxvar, level)
```

qNum	队列编号（对应 TKY_QUEUE_ID）
chNum	ADC 通道号或位掩码
hyswin	迟滞窗口，库内部软件实现迟滞比较器，用于按键消抖
chBaseline	参考基线，由上位机生成，仅参与决定最终应用中的触摸灵敏度
maxvar	参考最大变化量，与参考基线对应，由上位机生成，参与决定最终触摸灵敏度
level	灵敏度等级（1~10），值越小触摸越灵敏

2.4. 通道配置

配置的触摸通道由两部分组：

触摸队列（单通道检测）：

- 通过 ADC 通道号方式确定扫描通道
- 用于唤醒期间的按键扫描
- 对应配置中的前 `TKY_TOUCH_QUEUE_NUM` 个队列，在上述例子中对应队列 0-11

休眠队列（多通道组合检测）：

多通道组合检测是针对低功耗优化的功能，触摸模块内部支持多电极连接，可最大限度的减少睡眠状态下的能耗，配置方式参考例程代码。

- 采用位掩码方式选择扫描通道
- 芯片内部将选中通道连接在一起，可一次扫描多个通道
- 主要用于低功耗扫描，降低扫描时间
- 对应配置中的后 `TKY_SLEEP_QUEUE_NUM` 个队列，在上述例子中对应队列 12-23

2.5. 参数调整指南

2.5.1. 灵敏度调整

`Cfg` 文件中的参考基线和参考最大变化量是辅助灵敏度等级来生成最终触发阈值的。

基本概念：

- **参考基线**(`chBaseline`)：一般由上位机校准时生成，也可以手动填写，例如将某次上电时初始化代码自动调准出的基线替换进去。它的作用是表征触摸按键的寄生电容 C_b 。
- **参考最大变化量**(`maxvar`)：为当前参考基线下手指(或铜柱)用力充分与按键接触读到的变化量，表征手指触摸时引入的电容 C_f 。

对于一个确定的产品， C_b 和 C_f 相当于是固定的属性，则其变化率 η (%) 可以表示为：

$$\eta = \frac{C_f}{C_b} = \frac{\text{maxvar}}{\text{chBaseline}}$$

一般来说 η 大于 5%，我们认为是一个设计健康的触摸产品。

实际应用：

最终产品的装配误差，或者 PCB 生产的工艺误差可能会引起 C_b 和 C_f 略有波动。当我们设置了一个裕量比较充足的灵敏度，这个波动可以忽略。

由上述可知，**参考基线**和**参考最大变化量**的作用就是来确定手指引入的变化率，所以**参考基线**和**参考最大变化量**需要一一对应（在同一次上电中确定），可以自行从监控 log 中得到或者通过触摸调试上位机生成。一般确定后就不需要再动了，除非更换了产品外壳或者更换了不同型号的 PCB。

用户在保证**参考基线**和**参考最大变化量**正确的前提下，后续的触摸效果调试中不需要再考虑这两个参数，一般只需要调整灵敏度就足够了

触摸按键的触发最终还是库内部将灵敏度转化成触发阈值来判决的，下面将展示如何通

过灵敏度来计算最终触发阈值：

上述我们讲到 Cb 和 Cf 是一个确定产品的固有属性，但是电容值我们无法直观的看到，所以我们用参考基线和参考最大变化量来表征这两个值。由前面的公式我们可以得到

$$\eta = \frac{Cf}{Cb} = \frac{maxvar}{chBaseline} = \frac{value}{Baseline}$$

则有

$$value = \frac{maxvar * Baseline}{chBaseline}$$

而阈值 th 最终为

$$th = \frac{level}{10} * value = \frac{maxvar * Baseline}{chBaseline} * \frac{level}{10}$$

其中

- **Baseline**：实际应用中上电触摸通道初始化程序校准出来的基线值
- **value**：实际应用中的实际最大变化量
- **level**：灵敏度等级。

上述过程都是在库内部完成的，用户只需要在最开始修改 Level 参数（1~10），数值越小最终的阈值越低，灵敏度就越高

2.5.2. 抗干扰配置

- **TKY_FILTER_GRADE**：滤波器等级配置，越大抗干扰越强，但是按键响应会变迟钝。

2.5.3. 基线更新速度

基线更新速度由两个参数共同决定：

- **TKY_BASE_REFRESH_SAMPLE_NUM**：基线更新采样次数，触摸按键扫描 TKY_BASE_REFRESH_SAMPLE_NUM 次更新一次基线，步进为 1。
- **TKY_BASE_UP_REFRESH_DOUBLE**：基线从手指按下向着手指不按的方向恢复专用，例如上电初始化时候手指一直按下或者高温高湿测试结束从测试环境中拿出时
 - （1） 设置为 0 和 1 时不影响原有更新速度；
 - （2） 设置大于等于 2 时，基线恢复更新步进从 1 改为设置值
 - （3） 适用于上电时手指一直按下或高温高湿测试后恢复等需要快速恢复的场景

2.5.4. 典型参数

典型的核心配置参数

#define TKY_FILTER_MODE	3
#define TKY_FILTER_GRADE	2
#define TKY_BASE_REFRESH_ON_PRESS	0
#define TKY_BASE_UP_REFRESH_DOUBLE	10
#define TKY_BASE_DOWN_REFRESH_SLOW	0
#define TKY_BASE_REFRESH_SAMPLE_NUM	500

```

#define TKY_SHIELD_EN          1
#define TKY_SINGLE_PRESS_MODE  2
#define TKY_TOUCH_QUEUE_NUM    12
#define TKY_SLEEP_QUEUE_NUM    12
#define TKY_MAX_QUEUE_NUM      (TKY_TOUCH_QUEUE_NUM+TKY_SLEEP_QUEUE_NUM)
#define TKY_MAX_NOISE_CH_COUNT  3
#define TKY_CX_CH_IDX          13

```

典型的通道参数：

```

#define TKY_CHS_INIT          \
    GEN_TKY_CH_INIT( TKY_QUEUE_0, 9      , 10, 2072, 144, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_1, 8      , 10, 2083, 135, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_2, 6      , 10, 2084, 131, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_3, 10     , 10, 2091, 150, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_4, 7      , 10, 2067, 167, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_5, 5      , 10, 2052, 150, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_6, 1      , 10, 2097, 149, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_7, 3      , 10, 2076, 169, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_8, 4      , 10, 2056, 150, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_9, 0      , 10, 2082, 149, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_10, 2     , 10, 2060, 169, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_11, 11     , 10, 2068, 150, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_12, (1<<9 ), 10, 2072, 144, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_13, (1<<8 ), 10, 2083, 135, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_14, (1<<6 ), 10, 2084, 131, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_15, (1<<10), 10, 2091, 150, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_16, (1<<7 ), 10, 2067, 167, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_17, (1<<5 ), 10, 2052, 150, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_18, (1<<1 ), 10, 2097, 149, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_19, (1<<3 ), 10, 2076, 131, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_20, (1<<4 ), 10, 2056, 150, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_21, (1<<0 ), 10, 2082, 167, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_22, (1<<2 ), 10, 2060, 150, 5),\
    GEN_TKY_CH_INIT( TKY_QUEUE_23, (1<<11), 10, 2068, 149, 5)

```

以上述通道参数为例

参考基线：经上位机调试后生成

参考最大变化量：经上位机调试后生成，与参考基线对应

灵敏度等级：5（中等灵敏度）

迟滞窗口：例如配置为 10（最终库内部由灵敏度计算生成的触发阈值和释放阈值之间相差 10 个值）

3. 函数功能描述

3.1. TKY_BaseInit

uint8_t TKY_BaseInit(TKY_BaseInitTypeDef TKY_BaseInitStruct)

功能描述	该函数用于初始化触摸系统的基础模块，通过传入基础初始化结构体参数来配置系统的核心工作参数，通常用于系统上电后的首次初始化。
输入参数	TKY_BaseInitStruct，该参数类型为 TKY_BaseInitTypeDef。
返回值	初始化结果。 0x00：初始化成功； 0x01：滤波等级超出上限（最大为 16）。

初始化参数注解：

```
typedef struct
{
    uint8_t maxQueueNum;           //测试队列数量
    uint8_t singlePressMod;        //单按键模式
                                    //0-支持同时按下的多按键数量
                                    //1-最大值单按键
                                    //2-互斥单按键
    uint8_t shieldEn;              //屏蔽使能
    uint8_t filterMode;            //滤波器模式
    uint8_t filterGrade;           //滤波器等级
    uint8_t peakQueueNum;          //按键最大偏移队列
    uint8_t peakQueueOffset;       //按键最大偏移队列的偏移值
    uint8_t baseRefreshOnPress;     //基线在按键按下时是否更新
    uint8_t baseUpRefreshDouble;   //基线向上刷新倍速参数
    uint8_t baseDownRefreshSlow;   //基线向下更新降速参数
    uint8_t maxNoiseChCount;       //干扰通道数量
    uint8_t sleepChNum;            //睡眠通道数量
    uint32_t baseRefreshSampleNum; //基线刷新采样次数
    uint32_t *tkyBufP;             //测试通道数据缓冲区指针
}TKY_BaseInitTypeDef;
```

- **maxQueueNum**：使用的触摸按键通道数目，其值与实际使用的按键数目要严格一致，一般由触摸调试上位机 WCHTouchKeyTool 生成的配置头文件提供。
- **singlePressMod**：单按键模式开启使能，
0：多按键模式，多个按键按下时同时输出键值；
1：单按键模式，当多个按键同时按下时，输出变化量最大的键值；
2：互斥单按键模式，只有当前按键释放后才可以触发下一个按键。
- **shieldEn**：屏蔽功能使能，开启后可显著降低触摸的基础电容，从而提高变化率。
1：启动屏蔽功能；

0: 关闭屏蔽功能。

- **filterMode:** 选择要使用哪种滤波器模式，配置值参考库头文件，目前支持滤波器 3
- **filterGrade:** 滤波器等级，该参数可设置 0~15。该参数越大，检测结果越稳定，抗干扰能力越强，但是该参数越大，输出结果需要的样本数也越多，容易造成检测结果滞后或间隔非常短暂的按键值无法检测出来，所以需要根据实际应用中的需求，来设置该参数的大小。
- **peakQueueNum:** 按键最大偏移队列的队列号。该变量应用场景为——在 CS10V 测试，可设置为波动最大的通道值或最容易误触摸的通道值，若无测试条件，建议设置为手指按下时变化量最大的通道值。非 CS10V 滤波器使用时无需配置。
- **peakQueueOffset:** 按键最大偏移队列的偏移值。该变量应用场景同 peakQueueNum，在 CS10V 测试中，对 peakQueueNum 所设置的通道进行偏移修正，通常偏移值可以通过测试修改，通常尽可能赋较小数值。在无需进行 CS10V 测试的场景，该值可设置为 0。
- **baseRefreshOnPress:** 基线更新在按键按下时是否进行。
1: 触摸通道触发时继续按设定的间隔更新基线;
0: 触摸通道触发时，基线停止更新。
- **baseRefreshSampleNum:** 基线更新采样次数，触摸扫描每进行 baseRefreshSampleNum 次刷新一次基线。设置为 0 时，关闭基线刷新，参数越大，基线更新越缓慢。在同样的 baseRefreshSampleNum 设置值下，滤波器等级越高，基线刷新越慢。
- **baseUpRefreshDouble:** 基线向上更新的倍数参数，本质上增加基线向上刷新的步进，该参数范围 0~255，设置为 0 或 1 时不加速，如果对基线向上刷新和向下刷新有不同的要求时，尽量选择设置向下减速。
- **baseDownRefreshSlow:** 基线向下更新的减速参数，该参数范围为 0~255。该参数与 baseUpRefreshDouble 主要是为了解决基线向上更新和向下更新不同速的需求。具体请参考示例里的补充说明。
- **tkyBufP:** 测试通道数据缓冲区指针，由用户在外部分配，需要注意该缓冲区地址需要 4 字节对齐。

示例代码如下：

```
static void touch_Baseinit( void )
{

    TKY_BaseInitTypeDef TKY_BaseInitStructure = { 0 };
    if ( memcmp( TOUCH_VER_LIB, TOUCH_VER_FILE, strlen( TOUCH_VER_FILE ) ) )
    {
        PRINT( "head file error...\n" );
        while ( 1 );
    }
    //-----触摸按键基础设置初始化-----
    TKY_BaseInitStructure.filterMode      = TKY_FILTER_MODE;
    TKY_BaseInitStructure.shieldEn       = TKY_SHIELD_EN;
    TKY_BaseInitStructure.singlePressMod = TKY_SINGLE_PRESS_MODE;
```

```

    TKY_BaseInitStructure.filterGrade      = TKY_FILTER_GRADE;
    TKY_BaseInitStructure.maxQueueNum     = TKY_MAX_QUEUE_NUM;
    TKY_BaseInitStructure.maxNoiseChCount = TKY_MAX_NOISE_CH_COUNT;
    TKY_BaseInitStructure.sleepChNum      = TKY_SLEEP_QUEUE_NUM;
    TKY_BaseInitStructure.baseRefreshOnPress = TKY_BASE_REFRESH_ON_PRESS;
    TKY_BaseInitStructure.baseRefreshSampleNum = TKY_BASE_REFRESH_SAMPLE_NUM;
    TKY_BaseInitStructure.baseUpRefreshDouble = TKY_BASE_UP_REFRESH_DOUBLE;
    TKY_BaseInitStructure.baseDownRefreshSlow = TKY_BASE_DOWN_REFRESH_SLOW;
    TKY_BaseInitStructure.tkyBufP         = TKY_MEMBUF;
    sta                                   = TKY_BaseInit( TKY_BaseInitStructure );
    dg_log( "TKY_BaseInit:%02X\r\n", sta );
}

```

3.2. TKY_CHInit

Uint8_t TKY_CHInit(TKY_ChannelInitTypeDef TKY_CHInitStruct)

功能描述:	该函数用于初始化触摸系统的通道配置, 通过传入初始化结构体参数来配置通道的各项工作参数。用于系统启动时或通道参数需要重新配置时的初始化操作。
输入参数:	TKY_CHInitStruct, 该参数类型为 TKY_ChannelInitTypeDef。
返回值	初始化结果。 0x00: 初始化成功; Bit[1]: 转换队列编号超过最大队列数量; Bit[2]: 基线值溢出;

初始化参数注解:

```

typedef struct
{
    uint16_t queueNum;           //该通道在测试队列的序号
    uint16_t channelNum;        //该通道对应的 ADC 通道标号
    uint16_t hysteresisWindow;   //迟滞比较窗口
    uint16_t baseLine;          //参考基线
    uint16_t maxVar;            //参考最大变化量
    uint16_t sensitivityLevel;   //灵敏度 1~10
}TKY_ChannelInitTypeDef;

```

- **queueNum:** 该触摸按键在转换队列中的位置。该参数不得超过头文件中的 MAX_QUEUE_NUM 值, 同一个触摸按键可以占有转换队列的多个位置 (不允许在 CS10V 相关测试场景中重复占位)。
- **channelNum:** 触摸通道索引, 通常也是 ADC 通道编号。
- **hysteresisWindow:** 迟滞窗口, 用于按键消抖。
- **baseLine:** 参考基线值, 来自上位机生成的配置文件

- **maxVar**: 参考最大变化量, 其反应了手指引入的电容分量
- **sensitivityLevel**: 灵敏度等级 (1~10), 值越小触摸越灵敏。

示例代码如下:

```
// 定义并填充初始化结构体
TKY_ChannelInitTypeDef chInit = { TKY_QUEUE_0, 9, 10, 2284, 144, 5};
// 执行通道初始化
uint8_t result = TKY_CHInit(chInit);
if (result == 0) {
// 初始化成功, 通道可正常使用
}
```

3.3. TKY_GetCurChannelMean

```
uint8_t TKY_GetCurChannelData(uint16_t curChNum,
                               uint16_t CTransN,
                               uint16_t CTransCC,
                               uint16_t* scanArray,
                               uint16_t dataNum);
```

功能描述	该函数用于获取指定通道的采样数据, 支持多次转换, 以供后续噪声分析和数据处理。
输入参数	curChNum: 指定的触摸通道编号, 与芯片相关。 CTransN: 转换次数参数, 范围 1~1023。 CTransCC: 转换控制参数, CTransCC[1:0]——控制参数; CTransCC[15:2]——保留。 scanArray: 指向存储采样数据的缓冲区。 dataNum: 数据数量, 指定用于要获取的数据采样点数
输出参数	返回操作状态码 0: 成功获取通道数据 1: 采样数量为 0 2: 转换次数参数超出范围

3.4. TKY_GetCurQueueValue

```
uint16_t TKY_GetCurQueueValue(uint8_t curQueueNum)
```

功能描述	获取任意触摸通道滤波后的测量值。该函数可用于用户自行处理滤波后数据
输入参数	curQueueNum, 所选择的测量触摸通道队列编号, 注意并非 ADC 通道编号
输出参数	滤波模式 3 和 CS10 时为所选择的通道滤波后的变化值 (可直接和阈值比较)
输出参数	各通道按键值, 返回值各个位对应各个队列的按键, 例如队列 0 的触摸通道有按键, 对应最低位置 1。注意并非 ADC 通道编号

3.5. TKY_PollForFilterMode_3

uint16_t TKY_PollForFilterMode_3(void)

功能描述	该函数用于轮询并获取特定滤波模式 3 的各队列按键按下结果, 此函数采用阻塞轮询方式, 会占用 CPU 资源直至此次扫描结束
输入参数	无
输出参数	各通道按键值, 返回值各个位对应各个队列的按键, 例如队列 0 的触摸通道有按键, 对应最低位置 1。注意并非 ADC 通道编号

```
// 轮询滤波模式 3 按键输出
uint16_t keydata;
keydata = TKY_PollForFilterMode_3();

// 进行后续处理
if (keydata & KEY1_READY_BIT) {
    //KEY1 被按下
}
```

3.6. TKY_PollForFilterMode_CS10

uint16_t TKY_PollForFilterMode_CS10(void)

功能描述	该函数用于轮询并获取 CS10 的按键按下状态, 通过非阻塞方式获取滤波器的按键输出状态。此函数采用非阻塞方式轮询, 占用 CPU 资源小。需要高频调用, 最好放在主循环中调用, 否则影响按键响应的实时性。
输入参数	无
输出参数	各通道按键值, 返回值各个位对应各个队列的按键, 例如队列 0 的触摸通道有按键, 对应最低位置 1。注意并非 ADC 通道编号

3.7. TKY_ScanForWakeUp

uint16_t TKY_ScanForWakeUp(void)

功能描述	该函数用于扫描系统唤醒, 检测指定的唤醒条件是否满足。通常用于低功耗模式下定期检查唤醒条件, 以便及时恢复正常工作状态。
输入参数	无
输出参数	返回唤醒状态 0: 不满足唤醒条件 1: 唤醒条件满足 2: 睡眠通道配置出错

3.8. TKY_SetCurQueueSleepStatus

uint8_t TKY_SetCurQueueSleepStatus(uint8_t curQueueNum, uint8_t sleepStatus)

功能描述	该函数用于控制指定队列的睡眠状态，使该队列进入或退出睡眠模式。
输入参数	curQueueNum: 所选择的测量触摸通道队列编号，注意并非 ADC 通道编号。 sleepStatus: 所设置的休眠状态，为 0 不休眠，非 0 则休眠
输出参数	设置成功输出 0，队列序号超限输出 1

3.9. TKY_SetSleepStatusValue

void TKY_SetSleepStatusValue(uint16_t setValue)

功能描述	设置所有队列通道的休眠模式，与函数 TKY_SetCurQueueSleepStatus 相比，本函数是直接配置全部通道休眠状态，对应位为 0 则不休眠，为 1 则休眠
输入参数	setValue, 所有通道的休眠状态，对应位为 0 则不休眠，为 1 则休眠
输出参数	无

3.10. TKY_ReadSleepStatusValue

uint16_t TKY_ReadSleepStatusValue(void)

功能描述	设置所有队列通道的休眠模式，与函数 TKY_SetCurQueueSleepStatus 相比，本函数是直接配置全部通道休眠状态，对应位为 0 则不休眠，为 1 则休眠
输入参数	无
输出参数	返回所有通道的休眠状态，对应位为 0 则不休眠，为 1 则休眠

3.11. TKY_SetCurQueueChargeTime

**uint8_t TKY_SetCurQueueChargeTime(uint8_t curQueueNum,
uint16_t CTransN,
uint16_t CTransCC);**

功能描述	该函数用于配置指定队列的充电时间相关参数。
输入参数	curQueueNum: 所选择的测量触摸通道队列编号，注意并非 ADC 通道编号。 CTransN: 转换次数参数，范围 1~1023。 CTransCC: 转换控制参数, CTransCC[1:0]——控制参数; CTransCC[15:2]——保留。
输出参数	返回操作状态码 0: 成功设置充电时间参数 1: 队列编号超出范围 2: 转换次数参数超出范围

3.12. TKY_SetGroupCurQueueChargeTime

```
uint8_t TKY_SetGroupCurQueueChargeTime(uint8_t groupidx,
                                         uint8_t curQueueNum,
                                         uint16_t CTransN,
                                         uint16_t CTransCC)
```

功能描述	该函数给每个触摸队列配置最多四组充电参数。
输入参数	groupidx: 组索引 curQueueNum: 所选择的测量触摸通道队列编号, 注意并非 ADC 通道编号。 CTransN: 转换次数参数, 范围 1~1023。 CTransCC: 转换控制参数, CTransCC[1:0]—控制参数; CTransCC[15:2]—保留
输出参数	返回操作状态码 0: 成功设置充电时间参数 1: 队列编号超出范围 2: 转换次数参数超出范围 3: 组索引无效

3.13. TKY_GetCurQueueChargeTime

```
uint8_t TKY_GetCurQueueChargeTime(uint8_t curQueueNum,
                                     uint16_t* CTransN,
                                     uint16_t* CTransCC)
```

功能描述	该函数用于读取指定队列的当前充电时间参数, 包括转换次数和转换控制参数。可用于参数验证、状态监测或动态调整前的参数获取。
输入参数	curQueueNum: 所选择的测量触摸通道队列编号, 注意并非 ADC 通道编号。
输出参数	CTransN: 输出参数, 指向存储转换次数参数的缓冲区。 CTransCC: 输出参数, 指向存储转换控制参数的缓冲区。
返回值	返回操作状态码 0: 成功获取充电时间参数 1: 队列编号超出范围

3.14. TKY_GetGroupCurQueueChargeTime

```
uint8_t TKY_GetGroupCurQueueChargeTime(uint8_t curQueueNum,
                                         uint8_t groupidx,
                                         uint16_t* CTransN,
                                         uint16_t* CTransCC)
```

功能描述	该函数用于读取指定组内特定队列的当前充电时间参数, 包括转换次数和转换
------	-------------------------------------

	控制参数。
输入参数	curQueueNum: 所选择的测量触摸通道队列编号, 注意并非 ADC 通道编号。 Groupidx: 组索引
输出参数	CtransN: 输出参数, 指向存储转换次数参数的缓冲区。 CTransCC: 输出参数, 指向存储转换控制参数的缓冲区。
返回值	返回操作状态码 0: 成功获取充电时间参数 1: 队列编号超出范围 3: 组索引超出范围

3.15. TKY_SetCurQueueThreshold

```
uint8_t TKY_SetCurQueueThreshold(uint8_t curQueueNum,
                                   uint8_t threshold,
                                   uint8_t threshold2)
```

功能描述	该函数用于配置指定队列的阈值参数, 包括触发阈值和释放阈值
输入参数	curQueueNum: 所选择的测量触摸通道队列编号, 注意并非 ADC 通道编号。 Threshold: 该通道的上限门槛值, 即按键检测状态由无变有的判决值。 Threshold2: 该通道的下限门槛值, 即按键检测状态由有变无的判决值
返回值	返回操作状态码 0: 成功设置阈值参数 1: 列编号无效

3.16. TKY_GetCurQueueThreshold

```
uint8_t TKY_GetCurQueueThreshold(uint8_t curQueueNum,
                                   uint8_t *threshold,
                                   uint8_t *threshold2)
```

功能描述	该函数用于读取指定队列的阈值参数, 包括触发阈值和释放阈值。
输入参数	curQueueNum: 所选择的测量触摸通道队列编号, 注意并非 ADC 通道编号。
输出参数	Threshold: 指向存储上限阈值(触发阈值)的缓冲区。 Threshold2: 指向存储下限阈值(释放阈值)的缓冲区
返回值	返回操作状态或错误码 0: 成功获取阈值参数 1: 队列号超出范围

3. 17. TKY_GetCurIdleStatus

uint8_t TKY_GetCurIdleStatus(void)

功能描述	查询库内部状态机当前是否为空闲状态
输入参数	无
返回值	返回当前系统的空闲状态 0: 工作中 1: 空闲

3. 18. TKY_GetCurVersion

uint16_t TKY_GetCurVersion (void)

功能描述	获取当前触摸库版本号
输入参数	无
返回值	返回当前库版本号 例如返回值为 300, 则代表版本号为 3. 0. 0

3. 19. TKY_SetCurQueueBaseLine

uint8_t TKY_SetCurQueueBaseLine(uint8_t curQueueNum,
uint16_t baseLineValue);

功能描述	该函数用于设置指定队列的基线参值
输入参数	CurQueueNum: 所选择的测量触摸通道队列编号, 注意并非 ADC 通道编号 BaseLineValue: 要设置的基线值
返回值	返回操作状态或错误码 0: 设置成功 1: 队列号超出范围

3. 20. TKY_GetCurQueueBaseLine

uint8_t TKY_GetCurQueueBaseLine(uint8_t curQueueNum,
uint16_t* baseLineValue)

功能描述	获取指定队列通道的基线值
输入参数	curQueueNum: 所选择的测量触摸通道队列编号, 注意并非 ADC 通道编号

	baseLineValue: 输出参数, 指向存储基线值的缓冲区
返回值	返回操作状态或错误码 0: 获取成功 1: 队列号超出范围

3.21. TKY_GetCurQueueValue

```
uint8_t TKY_GetCurQueueValue(uint8_t curQueueNum,
                              uint16_t* queueValue)
```

功能描述	读取指定队列的当前数值, 是原始采样值经扫描滤波器处理后的数值, 适用于实时数据监测
输入参数	curQueueNum: 所选择的测量触摸通道队列编号, 注意并非 ADC 通道编号
输出参数	queueValue: 输出参数, 指向存储滤波值的缓冲区
返回值	返回操作状态或错误码 0: 成功获取 1: 队列编号超出范围

3.22. TKY_GetCurQueueRealVal

```
uint8_t TKY_GetCurQueueRealVal (uint8_t curQueueNum,
                                  uint16_t *queueValue);
```

功能描述	该函数用于读取指定队列的当前实时测量值, 用于实时监测、数据显示。
输入参数	curQueueNum: 所选择的测量触摸通道队列编号, 注意并非 ADC 通道编号
输出参数	queueValue: 输出参数, 指向存储真实值的缓冲区
返回值	返回操作状态或错误码 0: 成功获取 1: 队列编号超出范围

3.23. TKY_SetBaseRefreshSampleNum

```
void TKY_SetBaseRefreshSampleNum(uint8_t newValue)
```

功能描述	设置基线刷新采样次数, 每隔多少次采样刷新一次基线。为确保安全更新设置, 请查询空闲状态(TKY_GetCurIdleStatus), 在空闲时进行更新
输入参数	newValue, 新采样次数
输出参数	无

3.24. TKY_SetBaseUpRefreshDouble

```
void TKY_SetBaseUpRefreshDouble (uint8_t newValue)
```

功能描述	设置基线向上更新的倍数参数。为确保安全更新设置，请查询空闲状态 (TKY_GetCurIdleStatus)，在空闲时进行更新
输入参数	newValue，新采样次数
输出参数	无

3.25. TKY_SetBaseDownRefreshSlow

```
void TKY_SetBaseDownRefreshSlow (uint16_t newValue)
```

功能描述	设置基线向下更新的减速参数。为确保安全更新设置，请查询空闲状态 (TKY_GetCurIdleStatus)，在空闲时进行更新
输入参数	newValue，新采样次数
输出参数	无

3.26. TKY_SetFilterMode

```
void TKY_SetFilterMode (uint8_t newValue)
```

功能描述	设置新滤波模式。一般场景不建议使用，仅使用在休眠场景下特殊场景切换，具体参看低功耗休眠例程。
输入参数	newValue，新滤波模式
输出参数	无

3.27. TKY_ClearHistoryData

```
void TKY_ClearHistoryData (uint8_t curFilterMode)
```

功能描述	清除当前滤波器的历史数据。应用在触摸转换被较长时间打断，历史数据无意义的场景。
输入参数	curFilterMode，当前滤波模式，便于清除不同的滤波器。
输出参数	无

3.28. TKY_SaveAndStop

```
void TKY_SaveAndStop (void)
```

功能描述	保存触摸相关寄存器值，并且在判断触摸扫描空闲时暂停触摸功能，以腾出 ADC 模块用于 ADC 转换
输入参数	无
输出参数	无

3. 29. TKY_LoadAndRun

void TKY_LoadAndRun (void)

功能描述	载入触摸相关寄存器值，并重新启动被暂停的触摸按键功能
输入参数	无
输出参数	无

3. 30. TKY_NoiseAndMeanEstimate

```
uint8_t TKY_NoiseAndMeanEstimate(uint16_t *samples,
                                   uint32_t count,
                                   uint16_t* noise,
                                   uint16_t* mean);
```

功能描述	该函数通过对输入的样本数据进行统计分析，估计信号的噪声水平。同时计算样本的平均值，用于后续的信号处理分析。
输入参数	samples: 输入样本数据数组指针，指向待分析的原始采样数据 count: 样本数量，指定 samples 数组中的有效数据个数
输出参数	noise: 输出参数指针，用于返回所有样本的噪声水平 mean: 输出参数指针，用于返回所有样本的平均值
返回值	返回操作状态码 0: 成功 1: 队列编号无超出范围

3. 31. TKY_SetCurGroupNoiseThreshold

```
uint8_t TKY_SetCurGroupNoiseThreshold(uint8_t groupidx,
                                         uint16_t *noise_threshold,
                                         uint8_t dataNum);
```

功能描述	该函数通过对输入的样本数据进行统计分析，估计信号的噪声水平。同时计算样本的平均值，用于后续的信号处理分析。
输入参数	groupidx: 当前参数组索引 (0~3) noise_threshold: 输入噪声样本，按顺序与触摸队列序列对应 dataNum: 噪声样本数量，应与最大通道数量一致
返回值	返回操作状态码 0: 成功设置噪声阈值 1: 队列编号超出范围

3.32. TKY_GetCurQueueNoiseThreshold

```
uint8_t TKY_GetCurQueueNoiseThreshold(uint8_t curQueueNum,
                                       uint16_t* noise_threshold);
```

功能描述	该函数通过对输入的样本数据进行统计分析，估计信号的噪声水平。同时计算样本的平均值，用于后续的信号处理分析。
输入参数	curQueueNum: 指定队列编号
输出参数	noise_threshold: 指向存储噪声阈值的缓冲区
返回值	返回操作状态码 0: 成功获取噪声阈值 1: 队列编号超出范围

3.33. TKY_SetCurQueueSensitivityLevel

```
uint8_t TKY_SetCurQueueSensitivityLevel(uint8_t curQueueNum,
                                          uint8_t level);
```

功能描述	该函数用于配置指定队列的灵敏度级别，调整按键触发敏感度。
输入参数	curQueueNum: 当前队列编号 level: 灵敏度级别: 1~10, 为 0 表示灵敏度由库内部自行决定
返回值	返回操作状态码 0: 成功设置灵敏度级别 1: 队列编号超出范围 2: 设置灵敏度失败，噪声过大

3.34. TKY_GetCurQueueSensitivityLevel

```
uint8_t TKY_GetCurQueueSensitivityLevel(uint8_t curQueueNum,
                                          uint8_t* level);
```

功能描述	该函数用于读取指定队列的当前灵敏度级别设置，通过指针参数返回灵敏度值。
输入参数	curQueueNum: 当前队列编号
输出参数	Level: 输出参数，指向存储灵敏度级别的缓冲区
返回值	返回操作状态码 0: 成功获取灵敏度级别 1: 队列编号超出范围